INF221 — Lecture 5

Recorded lecture

Today's topics

1. Probabilistic analysis and randomized algorithms
   A. Randomized algorithms
   B. Generating random numbers
2. Review of Assignment 1 Solution (separate notebook)

Probabilistic analysis and randomized algorithms

Randomized algorithms

- For IDE-selection algorithm, random input provides much better performace than worst case ($\Theta(\ln n)$ vs $\Theta(n)$)
- We have shown that by analysing a *deterministic* algorithm using *probabilistic* methods
- Question: Could we avoid the worst case?
  - Worst case occurs when IDEs are evaluated in order from worst to best
  - We do not know beforehand how IDEs are ordered
  - But: if we randomize evaluation order, we will never be worse than $\Theta(\ln n)$
- Solution: *randomized algorithm*
  - Randomize evaluation order
  - Requires random permutation of $1, \ldots, n$

Randomly permuting an array in place

Algorithm

- Input: An array $A = \langle a_1, a_2, \ldots, a_n \rangle$ of numbers
- Output: Same array A, but with numbers a_i in random order

```
Randomize-in-place(A)
    n = A.length
    for i = 1 to n
        swap A[i] with A[Random(i, n)]
```

where `Random(i, n)` returns $k \in \{i, i+1, \ldots, n\}$ drawn with equal probability $p = 1/(n-i+1)$.

Run-time analysis

The algorithm requires

- n swaps
- n random numbers

Assuming that the run-time for `Random(i, n)` is constant, i.e., $\Theta(1)$, the run-time of the algorithm is thus $\Theta(n)$.

Proof of correctness

Permutations of a set

Let S be a finite set with n elements. Then any *ordered sequence* of all elements of S is a permutation of S.

In class exercise

Write down all permutations of the set $S = \{a, b, c\}$.

Number of permutations

A set S of $n = |S|$ elements has $n!$ permutations.

Proof

We build the permutation element by element from the set S. For the first element of the permutation we can choose between n elements, for the second between $n-1$ elements, and for the last element we have only 1 choice. Therefore, the total number of permutations is

$$n(n-1)(n-2)\cdots 2 \cdot 1 = n! \ .$$

k-permutation of a set

Let S be a finite set with n elements. Then any *ordered sequence of $k \leq n$ elements* selected from this set without replacement is a k-permutation of S. For given n and $k \leq n$, there are

$$\frac{n!}{(n-k)!}$$

k-permutations.

In class exercise

Write out all 2-permutations of the set $S = \{a, b, c\}$.

Proof of number of k-permutations in a set

We build the k-permutation element by element from set S. For the first element, we can choose between n elements, for the second between $n-1$, and so on, and for the kth element between $n-(k-1)=n-k+1$ elements. Therefore, we have

$$n(n-1)(n-2)\cdots(n-k+1)=\frac{n!}{(n-k)!}$$

possible k-permutations.

Aside: k-combinations of a set

Besides k-*permutations*, one often considers also k-*combinations* in combinatorics. A k-*combination* is any unordered subset of an n-element set S formed by drawing from S without replacement. There are

$$\frac{n!}{k!(n-k)!}=\binom{n}{k}$$

different k-combinations for an n-elements set.

Invariant

```
1: Randomize-in-place(A)
2:    n = A.length
3:    for i = 1 to n
4:        swap A[i] with A[Random(i, n)]
```

Immediately before the ith iteration of the loop on lines 3–4, the subarray $A[1..i-1]$ contains any of the possible $(i-1)$-permutations of the n elements in A with probability $(n-i+1)!/n!$.

Initialization

We have $i=1$, i.e., we consider the empty array $A[1..0]$. This is the only zero-permutation of A, and it occurs with probability 1, consistent with $(n-1+1)!/n!=n!/n!=1$. We have thus proven the initialization.

Maintenance

We assume that the invariant holds just before the ith iteration, i.e., $A[1..i-1]$ contains each of the possible $(i-1)$-permutations of A with equal probability $(n-i+1)!/n!$.

- With the next step, we generate a particular i-permutation $\langle x_1, x_2, \ldots, x_{i-1}, x_i\rangle$.
- Split this into $\langle x_1, x_2, \ldots, x_{i-1}\rangle$, and x_i
- Let E_1 be the event that the algorithm has placed our particular $(i-1)$-permutation $\langle x_1, x_2, \ldots, x_{i-1}\rangle$ in $A[1..i-1]$
- By induction hypothesis, $\Pr\{E_1\}=(n-i+1)!/n!$
- Let E_2 be the event that the algorithm places our particular x_i in $A[i]$

- The algorithm selects the new value in $A[i]$ at random from $A[i..n]$, whence $\Pr\{E_2\} = 1/(n-i+1)$
- Finding $\langle x_1, x_2, \ldots, x_{i-1}, x_i \rangle$ in $A[1..i]$ is equivalent to E_1 *and* E_2 happening together
- Thus, we need to obtain $\Pr\{E_1 \cap E_2\}$
- Using
  1. Bayes' theorem
  2. that we select $A[i]$ at random from $A[i..n]$, so that $\Pr\{E_2|E_1\} = \Pr\{E_2\}$
  3. and the induction hypothesis we have

$$\begin{aligned}
\Pr\{E_1 \cap E_2\} &= \Pr\{E_2|E_1\}\Pr\{E_1\} && (1)\\
&= \Pr\{E_2\}\Pr\{E_1\} && (2)\\
&= \frac{1}{n-i+1}\frac{(n-i+1)!}{n!} && (3)\\
&= \frac{(n-i)!}{n!}\ . && (4)
\end{aligned}$$

This is exactly the probability required by the invariant at the beginning of the $(i+1)^{\text{st}}$ iteration, so we have proven the maintenance of the induction hypothesis.

Termination

We now have $i = n+1$, whence the invariant states that $A[1..n]$ contains any of the possible n-permutations with probability $(n-(n+1)-1)!/n! = 0!/n! = 1/n!$. Since an n-permutation is a permutation of the entire array, and each possible permutation occurs with the same probability $1/n!$, we have proven that the algorithm generates uniform random permutations of A.

Generating random numbers

"Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin." — John von Neumann (1951)

Literature

- Classiscal treatment: Donald Knuth, The Art of Computer Programming, vol. 2
- Textbook: JE Gentle, Random number generation and Monte Carlo methods
- Summary: HE Plesser, Generating Random Numbers, in S. Grün, S. Rotter, *Analysis of Parallel Spike Trains*, 2010

Types of random number generators

Physical random number generators

- The only source of true randomness
- E.g. radioactive decay, shot noise in electronic circuits
- Main problem: low rate of random numbers
- Not much used in practice

Pseudo-random number generators

- Based on some mathematical approach to generating sequences of "unordered" numbers
- *Pseudo*-random, not truly random, since sequences are generated by deterministic algorithms
- Typically generate integers uniformly distributed on $0, 1, \ldots M$
- Challenges in creating PRNGs:
  - fast
  - long period
  - uniformity of distribution
  - no correlations between numbers
- Long considered a "dark art", but much better theoretical understanding and control over the past 20 years
- Most important classes
  - Recurrence-based generators
  - Cryptographic generators

Quasi-random number generators

- Carefully crafted random numbers to sample a space
- Used much for numerical integration in high-dimensional spaces
- Also known as low-discrepancy series
- Example: Sobol sequence
- Not discussed further here

Recurrence-based generators

- Most common type of PRNG for scientific applications/simulations
- Based on repeated application of mathematical rule
- At any time, the generator is fully described by *state variable* Σ
- **Seed** (initialize):

$$\Sigma_0 = \mathcal{S}(S)$$

  - seed S
  - seeding function $\mathcal{S}$
- **Iterate**:

$$\Sigma_j = \mathcal{I}(\Sigma_{j-1})$$

    - iteration function $\mathcal{I}$
- **Draw**:

$$r_j = \mathcal{D}(\Sigma_j)$$

    - r_j is r-th random number generated
    - drawing function $\mathcal{D}$
- For simple generators, Σ is an integer and $\mathcal{S}$ and $\mathcal{D}$ are the identity function

Example: Linear Congruential Generator (LCG)

- **State variable**: Σ, typically 32 bit integer
- **Seed function**: $\mathcal{S}(s) = s$
- **Drawing function**: $\mathcal{D}(\Sigma) = \Sigma$
- **Iteration function**: $\mathcal{I}(\Sigma) = (a\Sigma + c) \bmod m$
- Choice of constant a, c, m is **essential**
- Popular choice (Lewis et al, 1969)
    - $a = 7^5 = 16807$
    - $c = 0$
    - $m = 2^{31} - 1$
    - Largest number that can be generated: $M = m - 1 = 2^{31} - 2 = 2,147,483,646$
- For these choices, the generator has maximal *period* $\rho = M$
    - all numbers will be drawn before any number repeats

Python implementation

In [1]:
```
import matplotlib.pyplot as plt
%matplotlib inline
```

In [2]:
```
plt.rcParams['figure.figsize'] = (15, 5)
plt.rcParams['text.usetex'] = True
```

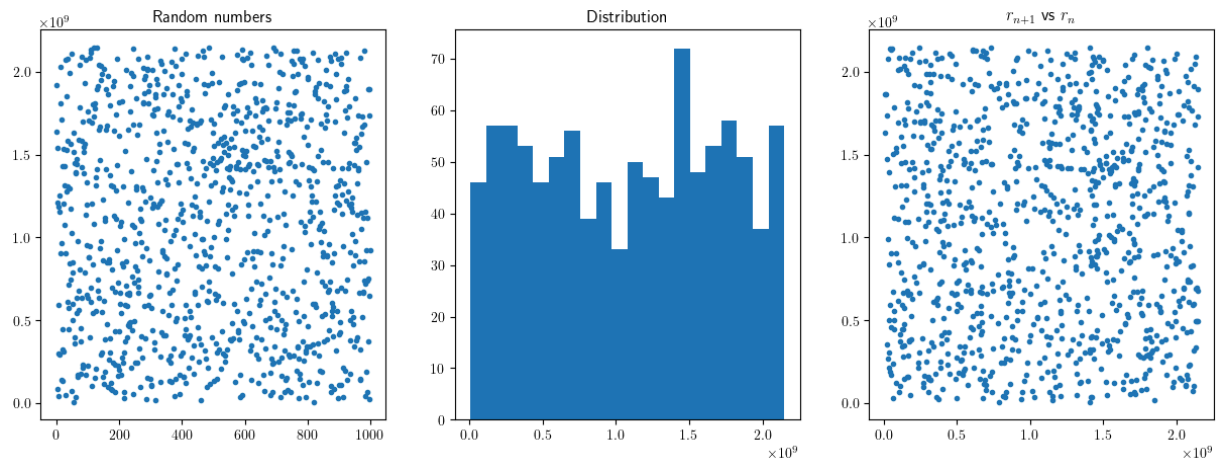
In [3]:
```
class LCG:
    a = 16807
    c = 0
    m = 2**31 - 1

    def __init__(self, seed):
        self.sigma = seed

    def draw(self):
        self.sigma = ( self.a * self.sigma + self.c ) % self.m
        return self.sigma
```

In [4]:
```
lcg = LCG(51223423)
r = [lcg.draw() for _ in range(1000)]
```

In [5]:
```
plt.subplot(1, 3, 1)
plt.plot(r, '.');
plt.title('Random numbers');
plt.subplot(1, 3, 2)
plt.hist(r, bins=20);
plt.title('Distribution');
plt.subplot(1, 3, 3)
plt.plot(r[:-1], r[1:], '.');
plt.title('$r_{n+1}$ vs $r_n$');
```



- Numbers seem unstructured and evenly distributed
- But: Short period
  - How quickly does a 3 GHz computer exhaust 2×10^9 random numbers?
  - Experience indicates that one should use only $\sqrt[3]{\rho} \approx 1290$ numbers to avoid correlations
- Also fails advanced tests for random number generators
- Note: Must be careful to avoid integer overflow when implementing $(a\Sigma + c) \bmod m$ in C, C++, Java, using 32 bit integers!

Multiple recursive generators

- Combine multiple recursion equations
- Recurrance based not only on previous state, but multiple earlier states
- Example: MRG32k3a (L'Ecuyer, 1999)

$$\Sigma_{1,n} = (1403580 \times \Sigma_{1,n-2} - 810728 \times \Sigma_{1,n-3}) \bmod m_1 \tag{5}$$
$$\Sigma_{2,n} = (527612 \times \Sigma_{2,n-1} - 1370589 \times \Sigma_{2,n-3}) \bmod m_2 \tag{6}$$
$$r_n = (\Sigma_{1,n} - \Sigma_{2,n}) \bmod m_1 \tag{7}$$
$$m_1 = 2^{32} - 209 = 4294967087 \tag{8}$$
$$m_2 = 2^{32} - 22853 = 4294944443 \tag{9}$$

- Two state variables Σ_1, Σ_2, each 53 bit
- Based on 3 previous Σ values
- Non-trivial draw function

- Period $\rho = (m_1^3 - 1)(m_2^3 - 1)/2 \approx 2^{191} \approx 10^{57}$
- Passes all test in TestU01 suite (2007)

Mersenne Twister

- Invented by Matsumoto and Nishimura (1998)
- So-called *feedback shift register* generator
- All arithmetic modulo 2, i.e., on bits
- State variable $\mathbf{\Sigma}$ is a vector of k 0s and 1s
- Iteration and drawing rules:

$$\mathbf{\Sigma}_j = \mathbf{A}\mathbf{\Sigma}_{j-1} \tag{10}$$
$$\mathbf{u}_j = \mathbf{B}\mathbf{\Sigma}_j \tag{11}$$
$$r_j = \sum_{l=1}^{w} u_{j,l} 2^{-l} \tag{12}$$

- Iteration matrix $\mathbf{A}$ is $k \times k$
- Output matrix $\mathbf{B}$ is $w \times k$, usually $w = 53 \ll k$
- Most-used variant has $k = 19937$
- Period $\rho = 2^{19937} - 1 \approx 10^{6000}$
- Passes almost all tests
- Widely used, e.g., in Python's `random` module

Permuted congruential generators

- Recent development (Melissa O'Neill, 2014)
- Solves some statistical problems in the Mersenne Twister and similar generators
- Much smaller state
- PCG64 generators is default in NumPy v1.17 and later

Cryptographic generators

- Problems with normal generators
  - Once state is know, we can predict all future random numbers
    - Extremely dangerous in any cryptographic conditions where unpredictability is essential
  - Can often not "skip ahead" in random number stream
    - Interesting in parallel simultions, where each of n processes wants to use only each n-th random number
- Cryptographic generators use standard encryption algorithms (e.g. AES)
- Algorithms use key K to encrypt plaintext P

$$C = E(K, P)$$

- We can generate random numbers by chosing seed S and drawing

$$C_j = E(K, S + j)$$

- Note that there is *no recurrence*: C_j does not depend on C_{j-1}

Advantages

- As long as key K is secret, we cannot predict future numbers from past numbers
- Skipping is trivial: we can obtain *any* number in the sequence at *any* time

Disadvantages

- Often slower than recurrent algorithms
  - But many newer CPUs provide hardware support for AES generators, making them fast
- Choice of K and S controls sequence of numbers
  - May require explicit rules for source of K and S to avoid manipulation
  - Applies also to recurvise algorithms (choice of seed)

A little more randomness: building a random number from bits

Given a random number generating function $\mathrm{Random}(0, 1)$, which returns 0 and 1 with equal probability, provide an algorithm $\mathrm{Random}(0, b)$ that returns uniformly distributed random integers from $\{0, 1, \ldots, b\}$, where $b = 2^n - 1$!

Correct solution building the random number as a binary number

```
Random(0, b)

r = 0
n = lg(b+1)
for k = 1..n
  r = r + RANDOM(0, 1) * 2^(k-1)
return r
```

Expected value of r

We first consider the problem mathematically. Each possible value of $r = \{0, 1, 2, \ldots b\}$ shall be selected with equal probability $p = 1/(b + 1)$. Therefore, the expected value of r is

$$E[r] = \sum_{r=0}^{b} rp = \frac{1}{b+1} \sum_{r=0}^{b} r = \frac{1}{b+1} \frac{b(b+1)}{2} = \frac{b}{2} \ . \tag{13}$$

Now we use indicator variables to check what the algorithm above does. We define

$$X_k = I\{\text{The } k\text{-th number returned by RANDOM(0,1) is 1}\} \ . \tag{14}$$

Then we have

$$r = \sum_{k=1}^{n} X_k 2^{k-1} \tag{15}$$

and thus

$$E[r] = E\left[\sum_{k=1}^{n} X_k 2^{k-1}\right] = \sum_{k=1}^{n} E[X_k] 2^{k-1} \tag{16}$$

$$= \frac{1}{2} \sum_{k=1}^{n} 2^{k-1} = \frac{1}{2} \sum_{k=0}^{n-1} 2^k \tag{17}$$

$$= \frac{2^n - 1}{2} = \frac{b}{2} \ , \tag{18}$$

where we have used the definition of b in the last step.

Proof of uniform distribution

The analysis above only proved the correctness of the mean of the numbers generated, but not their distribution. We will now see how to prove that all possible numbers are drawn with equal probability $p = 1/(b+1) = 1/2^n$.

Consider any integer m with $0 \leq m \leq b = 2^n - 1$, i.e., any number that may be returned by `RANDOM(0, b)`. This m has a *unique* binary representation

$$m = \sum_{k=1}^{n} m^{(k)} 2^{k-1} \ . \tag{19}$$

Using the X_k from above, we then have for the probablity to observe m

$$\Pr\{r = m\} = \Pr\{X_1 = m^{(1)} \wedge X_2 = m^{(2)} \wedge \cdots \wedge X_n = m^{(n)}\} \tag{20}$$

$$= \Pr\{X_1 = m^{(1)}\} \Pr\{X_2 = m^{(2)}\} \cdots \Pr\{X_n = m^{(n)}\} \tag{21}$$

$$= \prod_{k=1}^{n} \Pr\{X_k = m^{(k)}\} = \frac{1}{2^n} = \frac{1}{b+1} \ . \tag{22}$$

Here we used that the X_k are independent of each other so the joint probability can be expressed as product of the individual probabilities and that $\Pr\{X_k = m^{(k)}\} = 1/2$ by definition. Thus, we have proven that random numbers are equally distributed.

An incorrect solution

One could also consider the following algorithm

```
Random(0, b)
r = 0
for k = 1..b
  r = r + RANDOM(0, 1)
return r
```

This algorithm clearly can generate random numbers from 0 to b and one can show that $E[r] = b/2$ (using indicator random variables). But the distribution is not uniform as can be shown as follows: For $b = 2$, we execute the loop twice and obtain

- $r = 0$ if and only if `RANDOM(0, 1)` returns 0 both times,
- $r = 2$ if and only if `RANDOM(0, 1)` returns 1 both times,
- $r = 1$ if and only if either the first or the second call to `RANDOM(0, 1)` returns 1.

Therefore, there is exactly one way each to obtain $r = 0$ and $r = 2$, but two ways to obtain $r = 1$, so $r = 1$ will occur with greater probability than the other two values, violating the requirement to have a uniform distribution.

In general, the algorithm returns the number of heads obtained when flipping a fair coin b times. This corresponds to a sequence of $n = b$ Bernoulli trials with probability of success $p = \frac{1}{2}$ and the number of successes k follows the binomial distribution

$$P(X = k) = \binom{n}{k} \times \frac{1}{2^n}$$